

OOB as a metric

Out of Bag Errors

For bagged models in general, but for Random Forests in particular.

Why do we care about OOB errors?

I regularly teach a suite of ML and statistical models in an applied upper division statistics course. There are two reasons why I would like to harness the OOB errors.

1. OOB errors are more efficient than CV

If we can tune hyper parameters using OOB errors instead of CV, the model will be an order of magnitude more efficient.

2. OOB errors are part of the pedagogical process

One way to understand something is to understand it in contrast to another thing. Contrasting OOB and CV deepens the understanding of both metrics: how they are computed, used, and interpreted.

Example

```
library(tidyverse)
library(tidymodels)
library(palmerpenguins)
library(randomForest)
```

```

set.seed(47)
penguins <- penguins |>
  drop_na(body_mass_g)

penguin_split <- initial_split(penguins)
penguin_train <- training(penguin_split)
penguin_test <- testing(penguin_split)

penguin_bag_recipe <-
  recipe(body_mass_g ~ . ,
    data = penguin_train) |>
  step_unknown(sex, new_level = "unknown") |>
  step_mutate(year = as.factor(year)) |>
  step_naomit(all_predictors(), body_mass_g)

num_trees <- 500
penguin_bag <- rand_forest(mtry = 7,
  trees = num_trees) |>
  set_engine("randomForest", oob.error = TRUE) |> # note that we keep the OOB error!
  set_mode("regression")

penguin_bag_wflow <- workflow() |>
  add_model(penguin_bag) |>
  add_recipe(penguin_bag_recipe)

penguin_bag_fit <-
  penguin_bag_wflow |>
  fit(data = penguin_train)

```

Plotting OOB

Using `extract_fit_parsnip()` and `pluck()` we can pull out the relevant OOB metrics (either R^2 or MSE), but only as a function of **number of trees** (and not other tuning parameters).

(Tbh, I'm not 100% sure that the values plotted below are OOB, but I think they are because of the `oob.error = TRUE` argument above.)

```
penguin_bag_fit |>
  extract_fit_parsnip()
```

parsnip model object

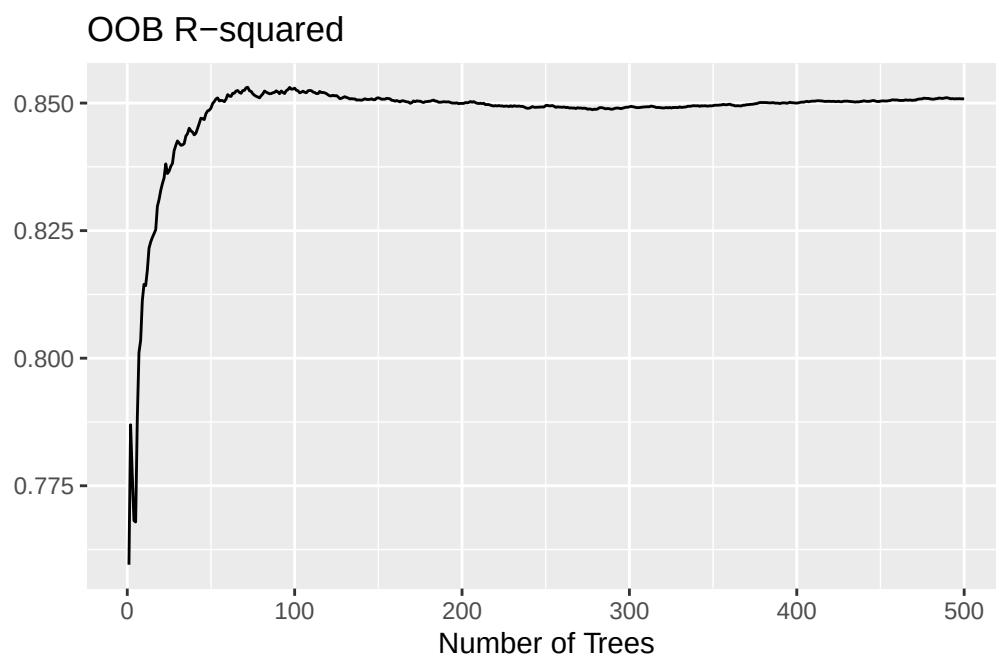
Call:

```
randomForest(x = maybe_data_frame(x), y = y, ntree = ~num_trees,      mtry = min_cols(~7, x)
              Type of random forest: regression
              Number of trees: 500
No. of variables tried at each split: 7

              Mean of squared residuals: 102398.6
              % Var explained: 85.08
```

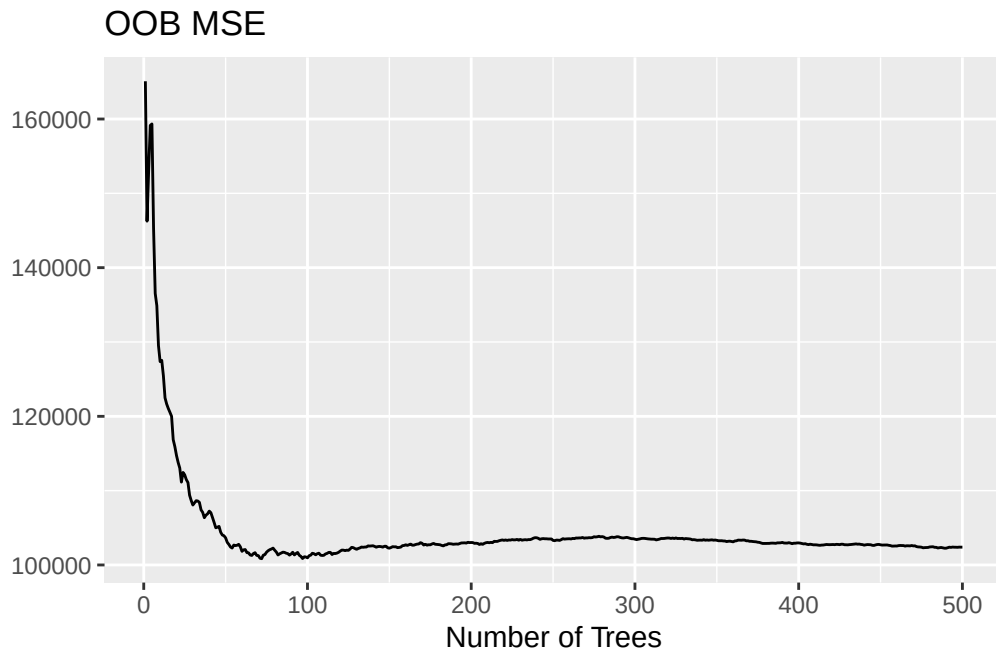
```
rsq <- penguin_bag_fit |>
  extract_fit_parsnip() |>
  pluck("fit") |>
  pluck("rsq")

data.frame(
  trees = 1:num_trees,
  rsq = rsq) |>
  ggplot() +
  geom_line(
    aes(x = trees, y = rsq)) +
  ylab("") +
  xlab("Number of Trees") +
  ggtitle("OOB R-squared")
```



```
MSE <- penguin_bag_fit |>
  extract_fit_parsnip() |>
  pluck("fit") |>
  pluck("mse")

data.frame(
  trees = 1:num_trees,
  MSE = MSE) |>
  ggplot() +
  geom_line(
    aes(x = trees, y = MSE)) +
  ylab("") +
  xlab("Number of Trees") +
  ggtitle("OOB MSE")
```



Model fitting using CV

We cannot easily, however, use OOB to do any of the model fitting. Consider the process below which selects `mtry` for a Random Forest model. The metric used is `rmse`, but the only way (as far as I can tell) to run the analysis is through cross validation. Because the OOB models have already been calculated, I would think that using CV is overkill and a waste of computational resources.

```
penguin_rf_recipe <-  
  recipe(body_mass_g ~ . ,  
    data = penguin_train) |>  
  step_unknown(sex, new_level = "unknown") |>  
  step_mutate(year = as.factor(year))  
  
penguin_rf <- rand_forest(mtry = tune(),  
  trees = tune()) |>  
  set_engine("ranger") |>  
  set_mode("regression")  
  
penguin_rf_wflow <- workflow() |>  
  add_model(penguin_rf) |>  
  add_recipe(penguin_rf_recipe)
```

```

set.seed(34)
penguin_folds <- vfold_cv(penguin_train,
                          v = 4)

penguin_grid <- grid_regular(mtry(range = c(2,7)),
                             trees(range = c(1,500)), levels = 5)

penguin_rf_tune <-
  penguin_rf_wflow |>
  tune_grid(resamples = penguin_folds,
            grid = penguin_grid)

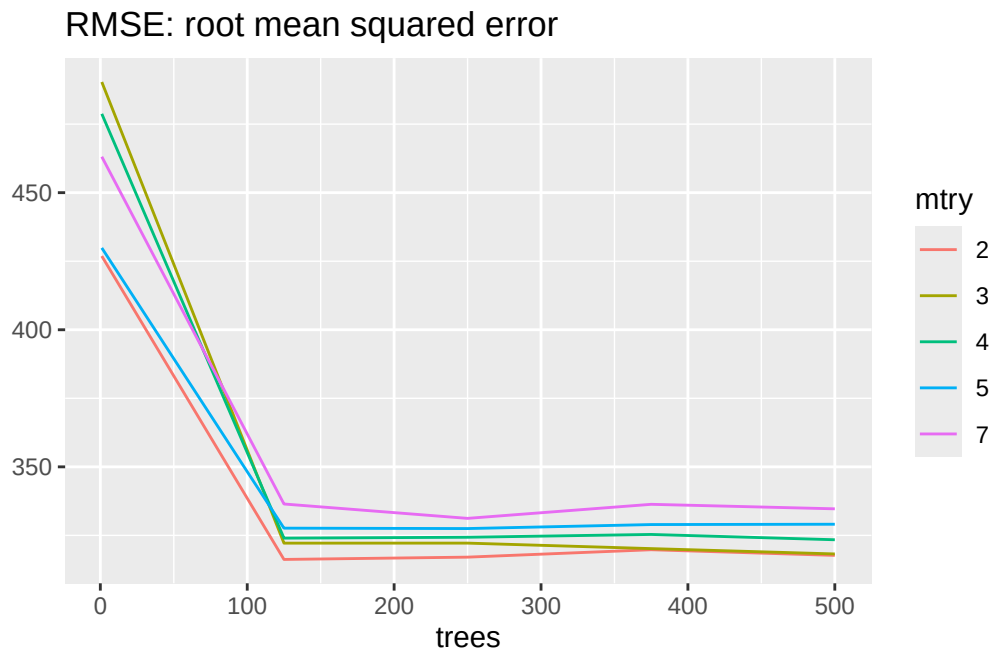
```

We use the `rmse` metric to consider the best `mtry`. Would be ideal to use the OOB `rmse` for such a task. In this case, it seems like `mtry` of 2 out performs the other `mtry` values, using cross validation.

```

penguin_rf_tune |>
  collect_metrics() |>
  filter(.metric == "rmse") |>
  mutate(mtry = as.factor(mtry)) |>
  ggplot() +
  geom_line(aes(x = trees, y = mean, color = mtry)) +
  labs(title = "RMSE: root mean squared error", y = "")

```



The point I'm trying to make is that cross validation is overkill when you have OOB information. The `rmse` based on the OOB models could be calculated when building the models, and so we wouldn't need to re-run everything 4 times for 4-fold cross validation.

Reporting metrics

Similarly, it would be nice to just be able to “pick” and report the best model using OOB errors. Would be ideal to have OOB `rmse` and OOB `rsq` as options here.

```
finalize_workflow(  
  penguin_rf_wflow,  
  select_best(penguin_rf_tune, metric = "rmse")) |>  
  last_fit(split = penguin_split) |>  
  collect_metrics()
```

```
# A tibble: 2 x 4  
  .metric .estimator .estimate .config  
  <chr>   <chr>       <dbl> <chr>  
1 rmse    standard      260.  pre0_mod0_post0  
2 rsq     standard       0.867 pre0_mod0_post0
```

Model fitting using OOB

@EmilHvitfeldt [wrote the function](#) `extract_oob()` (below), and it doesn't seem to be a perfectly parallel construction to `collect_metrics()`. Although, I think I can get the same results with some tidyverse wrangling.

How do we know that we have the OOB error? And is it `rmse`? Is there an equivalent R^2 ?

```
extract_oob <- function(x) {  
  x |>  
    extract_fit_engine() |>  
    pluck("prediction.error")  
}  
  
penguin_rf_tune_oob <-  
  penguin_rf_wflow |>  
  tune_grid(resamples = penguin_folds,  
            grid = penguin_grid,  
            control = control_grid(extract = extract_oob)) |>
```

```

unnest(.extracts) |>
unnest(.extracts)

penguin_rf_tune_oob

```

```

# A tibble: 100 x 8
  splits          id   .metrics .notes   mtry trees .extracts .config
  <list>         <chr> <list>  <list>   <int> <int>   <dbl> <chr>
1 <split [192/64]> Fold1 <tibble> <tibble>    2     1  177287. pre0_mod01_po~
2 <split [192/64]> Fold1 <tibble> <tibble>    2    125  102010. pre0_mod02_po~
3 <split [192/64]> Fold1 <tibble> <tibble>    2    250  100860. pre0_mod03_po~
4 <split [192/64]> Fold1 <tibble> <tibble>    2    375  102235. pre0_mod04_po~
5 <split [192/64]> Fold1 <tibble> <tibble>    2    500  101441. pre0_mod05_po~
6 <split [192/64]> Fold1 <tibble> <tibble>    3     1  187441. pre0_mod06_po~
7 <split [192/64]> Fold1 <tibble> <tibble>    3    125  101817. pre0_mod07_po~
8 <split [192/64]> Fold1 <tibble> <tibble>    3    250  101380. pre0_mod08_po~
9 <split [192/64]> Fold1 <tibble> <tibble>    3    375  100390. pre0_mod09_po~
10 <split [192/64]> Fold1 <tibble> <tibble>    3    500  101186. pre0_mod10_po~
# i 90 more rows

```

I think I would parse the code (ala `collect_metrics()`) using:

```

penguin_rf_tune_oob |>
  group_by(mtry, trees) |>
  summarize(mean_oob = mean(.extracts), .groups = "drop") |>
  arrange(mean_oob)

```

```

# A tibble: 25 x 3
  mtry trees mean_oob
  <int> <int>   <dbl>
1     2   250  100444.
2     2   500  101000.
3     2   375  101415.
4     3   250  101768.
5     3   500  102154.
6     3   375  102374.
7     2   125  103435.
8     3   125  103625.
9     4   375  104590.
10    4   250  105194.
# i 15 more rows

```